UNIVERSITAT JAUME I

SJK009 — Collaborative Robots

Person Follower Robot

Sensor Fusion with RGB, Depth, LiDAR, Odometry and Kalman Filter

Technical Documentation — ROS 2 / Webots Simulation

Author: Miguel Rebolledo

Professors: Enric Cervera Mateu

Ángel Juan Durán Bosch

Final Document — May 2026

Contents

1 Project Overview

This document describes the design and implementation of a **person-following robot** running on a TurtleBot3 Burger platform [3] inside a Webots R2025a simulation [2], controlled through a ROS 2 Humble node [1].

The system detects a target pedestrian, estimates their position by fusing four independent sensing modalities, predicts future motion with a Kalman filter, and commands the robot to follow the person while avoiding obstacles.

1.1 Goals

- Detect and continuously track a specific person among multiple pedestrians.
- Maintain a safe following distance of approximately 0.8 m.
- Recover tracking automatically after temporary occlusion or loss of sight.
- Avoid collisions using LiDAR-based obstacle detection.

1.2 Software Stack

Component	Technology
Simulation	Webots R2025a
Middleware	ROS 2 Humble
Object detection	YOLOv8n (Ultralytics)
Object tracking	BoTSORT
Depth estimation	Range Finder (simulated)
Laser scanning	LDS-01 LiDAR
Localisation	Odometry + ArUco markers
State estimation	2-D Kalman Filter
Language	Python 3

2 Simulation Environment

The world file `turtlebot3_burger_pedestrian_simple.wbt` defines an indoor office-like environment (5×10 m floor) with walls, a door, and four ArUco marker panels mounted at known world positions.

2.1 Robot Configuration

The TurtleBot3 Burger is equipped with the following sensors (defined in the `.wbt` world and exposed through the URDF plugin):

Sensor	ROS topic	Resolution / Rate
RGB camera	`/TurtleBot3Burger/camera/image_color`	640×480, FOV 64°
Range finder	`/TurtleBot3Burger/range_finder/range_image/image`	640×480 depth, 10 Hz
LiDAR (LDS-01)	`/scan`	360 rays, 5 Hz
IMU	`/imu`	20 Hz
Odometry	`/odom`	continuous

2.2 ArUco Markers

Four ArUco markers [12, 13] (DICT_4X4_50, IDs 0–3, size 0.3 m) are placed at fixed world coordinates to provide absolute localisation backup:

ID	x (m)	y (m)	z (m)
0	7.5	5.0	1.2
1	8.0	-5.0	1.2
2	10.0	2.0	1.2
3	10.0	-2.0	1.2

The PNG textures are generated by `webots/generate_aruco_markers.py`, which applies a horizontal mirror flip to compensate for Webots' internal texture inversion.

3 System Architecture

3.1 ROS 2 Node Graph

The entire behaviour is implemented in a single ROS 2 node (`PersonFollower`), which subscribes to sensor data and publishes velocity commands at 10 Hz.

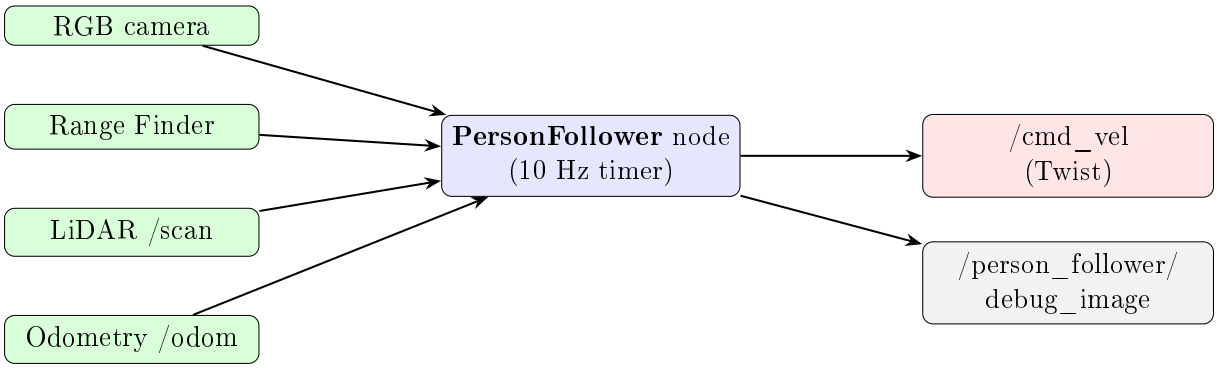

Figure 1: High-level ROS 2 node graph.

3.2 Processing Pipeline per RGB Frame

Each incoming RGB frame triggers the following pipeline:

4 Software Modules

4.1 Camera Model and Intrinsics

The simulated camera has a horizontal field-of-view of $\text{FOV}_H = 1.117$ rad ($\approx 64°$) at resolution 640×480. The focal length in pixels is derived analytically:

$$f_x = \frac{W/2}{\tan(\text{FOV}_H/2)} \approx 554 \text{ px} \tag{1}$$

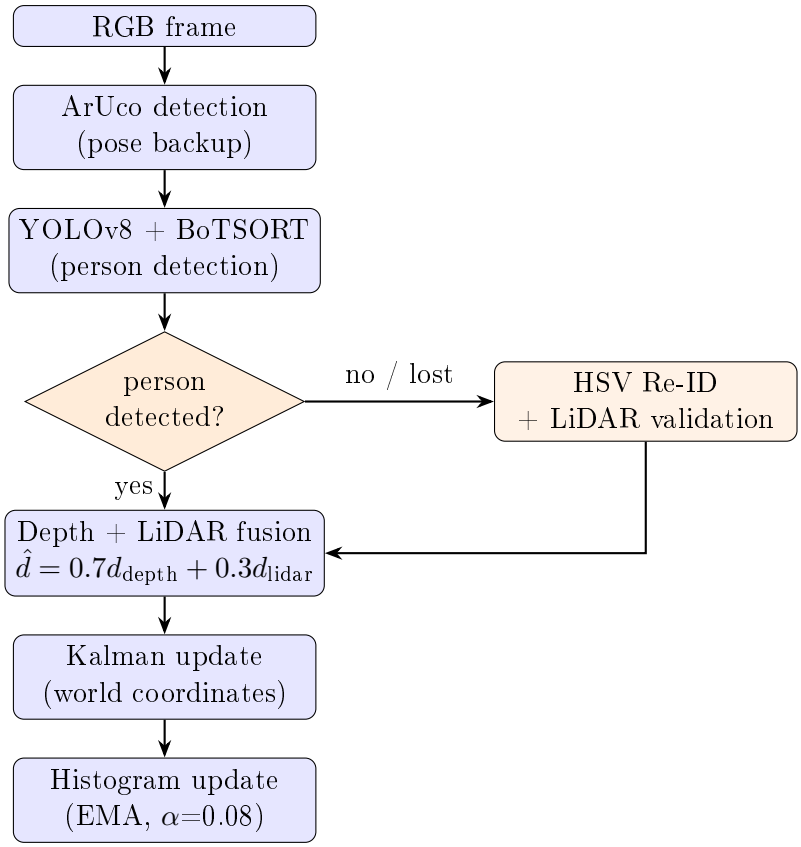

Figure 2: RGB processing pipeline executed on each camera frame.

The camera matrix used for ArUco pose estimation, computed with the OpenCV library [4], is:

$$K = \begin{pmatrix} f_x & 0 & W/2 \\ 0 & f_x & H/2 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 554 & 0 & 320 \\ 0 & 554 & 240 \\ 0 & 0 & 1 \end{pmatrix} \tag{2}$$

Lens distortion is modelled as zero, consistent with the ideal pinhole camera used in Webots.

4.2 Kalman Filter (`KalmanTracker`)

A constant-velocity Kalman filter [14, 15] operates in 2-D world coordinates to smooth and predict the pedestrian's position. The filter equations are implemented with NumPy [5].

State vector

$$\mathbf{x} = [x,\ y,\ v_x,\ v_y]^\top$$

Transition matrix (variable Δt)

$$F = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Observation model

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Only position (x, y) is observed; velocities are inferred from the filter.

Noise matrices

$$Q = \mathrm{diag}(0.05,\ 0.05,\ 0.20,\ 0.20), \quad R = 0.3 \cdot I_2$$

Prediction step

$$\hat{\mathbf{x}}^- = F\,\mathbf{x} \tag{3}$$
$$P^- = F\,P\,F^\top + Q \tag{4}$$

Update step

$$\mathbf{y} = \mathbf{z} - H\,\hat{\mathbf{x}}^- \tag{5}$$
$$S = H\,P^-\,H^\top + R \tag{6}$$
$$K = P^-\,H^\top\,S^{-1} \tag{7}$$
$$\mathbf{x} \leftarrow \hat{\mathbf{x}}^- + K\,\mathbf{y} \tag{8}$$
$$P \leftarrow (I - K\,H)\,P^- \tag{9}$$

Future position prediction

The method `get_predicted(t_ahead)` returns a forward-extrapolated estimate used to navigate towards the target when vision is temporarily lost:

$$(x_{\text{pred}},\ y_{\text{pred}}) = (x + v_x\,\tau,\ \ y + v_y\,\tau), \quad \tau = 0.8\ \text{s}$$

4.3 LiDAR Processing

Obstacle detection

All LiDAR rays within $\pm 30°$ of the forward direction are scanned for finite readings; the minimum value is stored as `obstacle_front`. This is used in the control law to modulate speed.

Leg cluster detection (`_detect_leg_clusters`)

Consecutive LiDAR points closer than 0.15 m (gap threshold) are grouped. This clustering is a lightweight heuristic variant of laser-based people detection [11]. A cluster is accepted as a potential human leg if:

- It contains between 3 and 12 points.
- Its bounding box width is less than 0.35 m.
- Its range is less than 2.5 m.

The centroid of each accepted cluster is returned in the robot frame. These centroids are used for two purposes: (1) validating YOLO detections, and (2) providing an independent distance estimate.

4.4 Person Detection and Tracking (`_process_yolo`)

1. **YOLO inference.** `YOLOv8n` [6, 7] runs with `persist=True` and the `botsort.yaml` [8, 9] tracker, returning bounding boxes with stable integer track IDs for class 0 (person) with confidence ≥ 0.30.
2. **Target acquisition.** On first detection, the robot locks onto the largest bounding box (by area), stores its track ID, and computes an initial HSV appearance histogram.
3. **Target maintenance.** On subsequent frames the locked ID is searched in the result list. If found, the bounding box is used directly.
4. **Re-identification (Re-ID).** If the target ID disappears for more than 2 s, each currently visible person is compared against the stored HSV appearance histogram [10] via histogram correlation. The best candidate is accepted if its similarity exceeds 0.75 *and* a LiDAR cluster exists within $\pm 25°$ of its image position.
5. **Histogram maintenance.** While the target is at 1–3 m, the stored histogram is updated with an exponential moving average:

$$H \leftarrow 0.92\, H + 0.08\, H_{\text{new}}$$

4.5 Distance Fusion

Once a bounding box is confirmed, the target distance $\hat{d}$ is computed by fusing the depth camera and the LiDAR:

1. **Depth camera.** A 25%-inset ROI of the bounding box is sampled from the `32FC1` depth image. The median of all valid pixels ($0.1 < d < 10$ m) is taken as d_{depth}.
2. **LiDAR.** The nearest leg cluster within $\pm 15°$ of the target heading gives d_{lidar}.
3. **Weighted fusion:**

$$\hat{d} = \begin{cases} 0.7\, d_{\text{depth}} + 0.3\, d_{\text{lidar}} & \text{both available} \\ d_{\text{depth}} & \text{depth only} \\ d_{\text{lidar}} & \text{LiDAR only} \\ \text{None} & \text{neither available} \end{cases}$$

4. **World-coordinate conversion.** When a fused distance is available it is converted to world coordinates and fed to the Kalman filter:

$$t_x = r_x + \hat{d}\cos(\psi + \theta), \quad t_y = r_y + \hat{d}\sin(\psi + \theta)$$

where (r_x, r_y) is the robot position from odometry, ψ is the robot yaw, and θ is the horizontal bearing to the target derived from the image column:

$$\theta = \left(\frac{c_x}{W} \cdot 2 - 1\right) \cdot \frac{\text{FOV}_H}{2}$$

5 Control Law

The control callback runs at 10 Hz and selects one of three operating modes.

5.1 Obstacle Speed Factor

Before any movement command is issued, a speed scaling factor is computed from the frontal LiDAR reading:

$$s = \begin{cases} 0 & d_{\text{front}} < 0.40 \text{ m} \\ 0.3 + 0.7\,\dfrac{d_{\text{front}} - 0.40}{0.65 - 0.40} & 0.40 \le d_{\text{front}} < 0.65 \text{ m} \\ 1 & d_{\text{front}} \ge 0.65 \text{ m} \end{cases} \tag{10}$$

5.2 Mode 1: Direct Following (Person Visible)

When both θ (bearing) and $\hat{d}$ (distance) are known:

$$\omega = \text{clip}(-1.4\,\theta,\ [-\omega_{\max},\ \omega_{\max}]) \tag{11}$$

$$f_a = \max\left(0,\ 1 - \frac{|\theta|}{\pi/4}\right) \tag{12}$$

$$v = s \cdot \text{clip}\Big(0.5\,(\hat{d} - d_{\text{des}}) \cdot f_a,\ [-v_{\max},\ v_{\max}]\Big) \tag{13}$$

where $d_{\text{des}} = 0.8$ m, $v_{\max} = 0.15$ m/s, $\omega_{\max} = 0.6$ rad/s. If $\hat{d} < d_{\text{des}}$ the robot backs up at -0.10 m/s. The factor f_a reduces forward speed when the robot is turning sharply, preventing it from driving past the target.

Mode 1b: Bearing only (no distance)

When only θ is available, the robot rotates to face the target and advances at maximum speed scaled by f_a and s.

5.3 Mode 2: Kalman Dead-Reckoning

When the person is not currently visible but the Kalman filter is initialised, the robot navigates to the predicted position $(\hat{x}, \hat{y})$:

$$d_{\text{goal}} = \sqrt{(\hat{x} - r_x)^2 + (\hat{y} - r_y)^2} \tag{14}$$

$$\alpha_{\text{goal}} = \text{atan2}(\hat{y} - r_y,\ \hat{x} - r_x) - \psi \tag{15}$$

$$v = \min(v_{\max} \cdot s,\ 0.3\,d_{\text{goal}}) \tag{16}$$

$$\omega = \text{clip}(0.8\,\alpha_{\text{goal}},\ [-\omega_{\max},\ \omega_{\max}]) \tag{17}$$

If the robot is within 0.5 m of the predicted point or has been in Kalman mode for more than 8 s without regaining sight of the target, it switches to a slow rotation to search (Mode 3) and resets the tracker state.

5.4 Mode 3: Search Rotation

The robot spins in the last observed angular direction at 0.3 rad/s until a person is detected again.

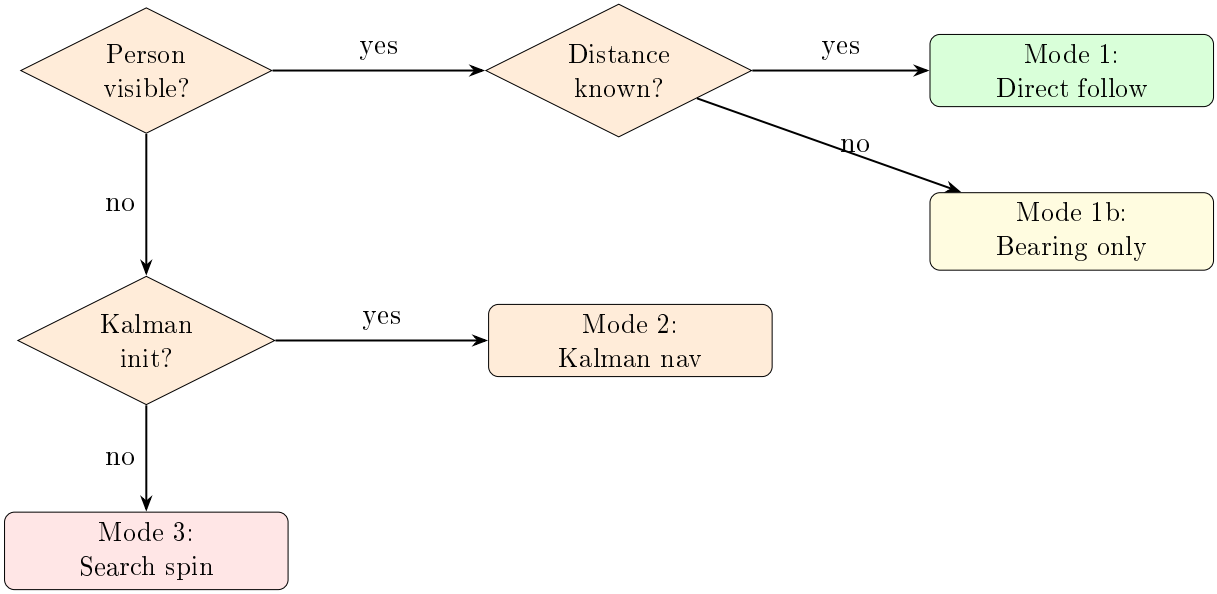

Figure 3: Control mode selection logic.

6 ArUco Localisation Backup

At every RGB frame, the ArUco detector (`DICT_4X4_50`) searches for known markers. For each detected marker with known world position $\mathbf{m}_w$, the pose of the robot is estimated as:

$$d = \|\mathbf{t}\|, \quad \alpha = \text{atan2}(t_x, t_z) \tag{18}$$

$$\psi_m = \text{atan2}(-n_x, n_z) \tag{19}$$

$$\phi = \psi_m + \alpha - \pi \tag{20}$$

$$r_x = m_{w,x} + d\cos\phi, \quad r_y = m_{w,y} + d\sin\phi \tag{21}$$

where $\mathbf{t}$ is the translation vector from the marker pose estimation and $\mathbf{n} = R_{cm}[:, 2]$ is the marker's normal direction. This alternative robot position estimate can be used to detect and correct odometry drift (currently logged at DEBUG level).

7 Debug Visualisation

The node publishes a debug image on `/person_follower/debug_image` combining:

- Bounding boxes for all detected persons (green for the target, red for others) with track IDs.
- Orange dots along the bottom edge indicating the projected positions of LiDAR leg clusters.

- A text overlay showing obstacle distance, Kalman velocity estimate, and current mode (`SIGUIENDO` / `RECUPERANDO`).

8 File Structure

```
person_follower/
+-- person_follower/
|   +-- __init__.py
|   +-- person_follower.py        # Main ROS 2 node (perception + control)
+-- webots/
|   +-- turtlebot3_burger_pedestrian_simple.wbt   # Simulation world
|   +-- turtlebot3_burger_pedestrian_no_walls.wbt # Alternative world
|   +-- generate_aruco_markers.py                 # ArUco PNG generator
|   +-- record_experiment.bash                    # rosbag recording helper
|   +-- aruco_0.png ... aruco_3.png               # Marker textures
|   +-- config.rviz                               # RViz configuration
+-- report/
|   +-- person_follower_report.tex   # This document
|   +-- plot_trajectories.py         # Trajectory plotting / analysis script
|   +-- traj_*.png                   # Generated trajectory figures
|   +-- sim_*.png                    # Simulation screenshots
+-- yolov8n.pt                     # Pre-trained YOLOv8 nano weights
+-- package.xml                    # ROS 2 package manifest
+-- setup.py / setup.cfg           # Python package setup
+-- README.md                      # Build and run instructions
+-- .gitignore
```

9 Parameter Summary

Parameter	Value	Description
`desired_distance`	0.80 m	Target following distance
`max_linear_speed`	0.15 m/s	Maximum forward/backward speed
`max_angular_speed`	0.60 rad/s	Maximum rotation speed
`OBSTACLE_STOP`	0.40 m	Full stop threshold
`OBSTACLE_SLOW`	0.65 m	Speed ramp threshold
`KALMAN_TIMEOUT`	8.0 s	Max time in Kalman-only mode
`REID_MIN_LOST_SECS`	2.0 s	Min lost time before Re-ID
`HIST_MATCH_THRESHOLD`	0.75	HSV correlation threshold
`ARUCO_MARKER_SIZE`	0.30 m	Physical marker side length
Depth-LiDAR fusion weight	70%/30%	Depth preferred over LiDAR

10 Simulation Evidence

This section presents screenshots taken during live simulation runs in Webots R2025a, demonstrating the end-to-end operation of the person-follower system.

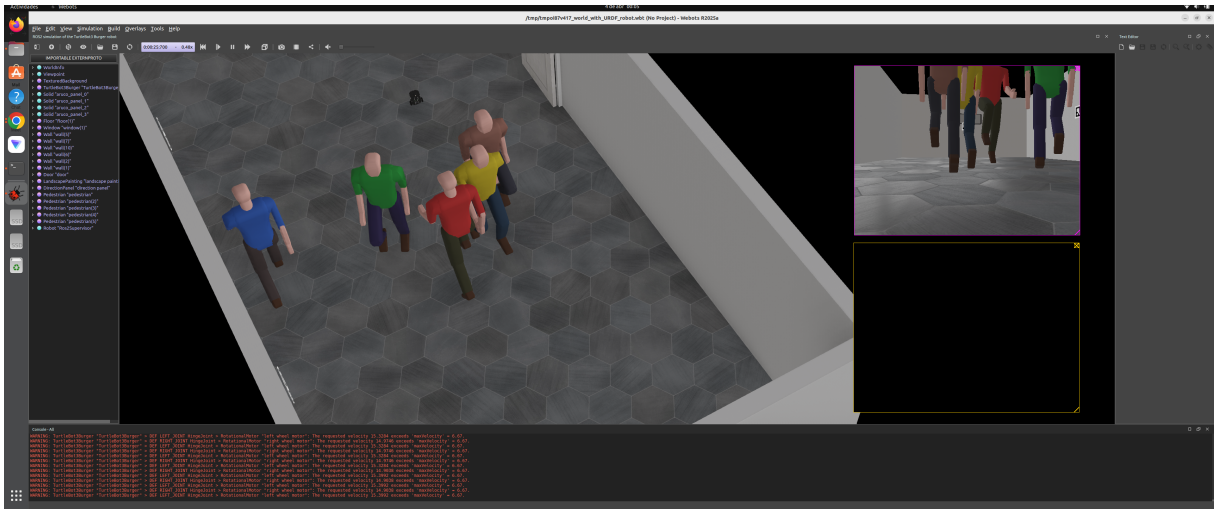

Figure 4: Webots simulation world with multiple pedestrians. The top-right inset shows the robot's onboard RGB camera stream. Several animated pedestrians move through the indoor environment while the TurtleBot3 Burger (visible in the centre of the scene) prepares to track a target.

10.1 Simulation World and Pedestrians

10.2 Active Tracking with Kalman Filter

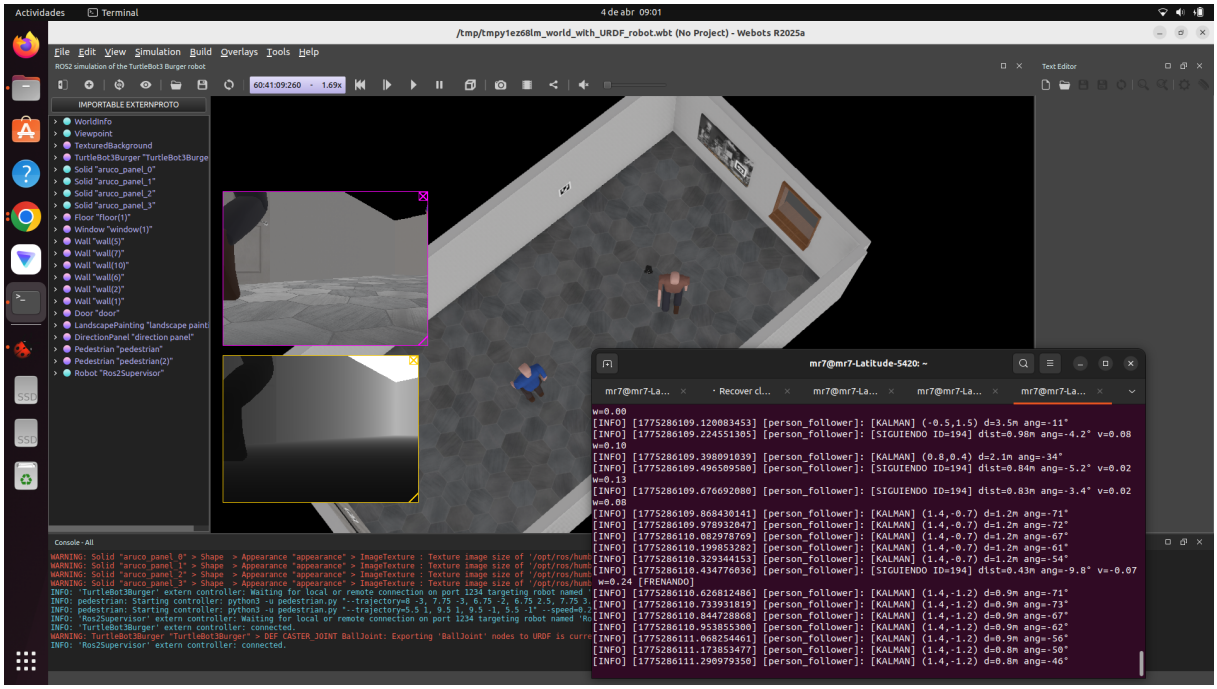

Figure 5: Robot actively following the locked target. The terminal on the right shows ROS 2 log output from the `person_follower` node: `[SIGUIENDO ID=94]` entries confirm that the YOLO+BoTSORT pipeline has acquired a stable track, and `[KALMAN]` lines show the Kalman filter operating in dead-reckoning mode during momentary occlusions. Distance and bearing values are updated at 10 Hz.

10.3 Active ROS 2 Topics

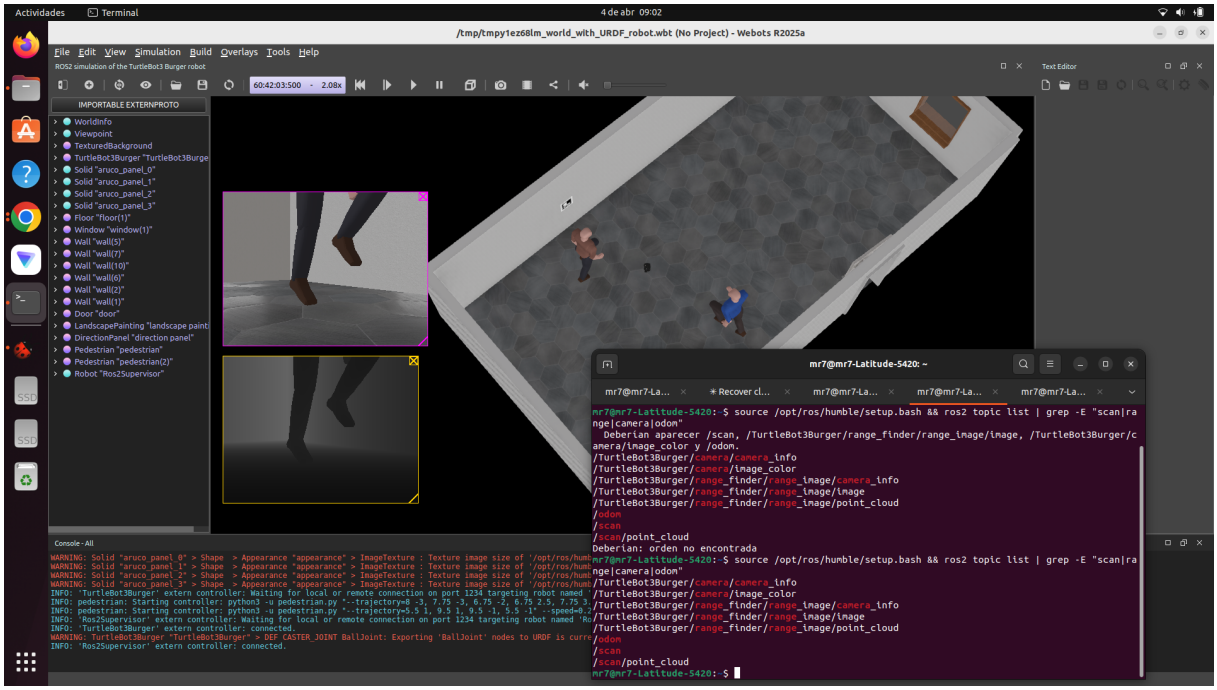

Figure 6: ROS 2 topic listing during a live run. All expected sensor streams are present: `/scan` (LiDAR), `/TurtleBot3Burger/camera/image_color` (RGB), `/TurtleBot3Burger/range_finder/range_image/image` (depth), and `/scan/point_cloud`. The camera stream also exposes a `/image_color` sub-topic used internally by the node.

10.4 Debug Image and Depth View

11 Trajectory Analysis

This section evaluates the behaviour of the system by plotting, in the Webots world frame, the path travelled by the robot together with the ground-truth paths of the pedestrians and the target position estimated by the perception pipeline.

11.1 Data Sources

Three independent sources of trajectory data are combined:

- **Robot trajectory.** The wheel odometry published on `/odom` provides the robot pose throughout the run.
- **Perceived target trajectory.** The node publishes the fused measurement and the Kalman estimate of the target on the topics `/person_follower/target_measurement` and `/person_follower/target_estimate` (both of type `geometry_msgs/PointStamped`). Recording these together with `/odom` makes the rosbag self-contained for the analysis.
- **Pedestrian ground truth.** The Webots `Pedestrian` nodes follow deterministic way-point lists declared in the `.wbt` world file through the `--trajectory` controller

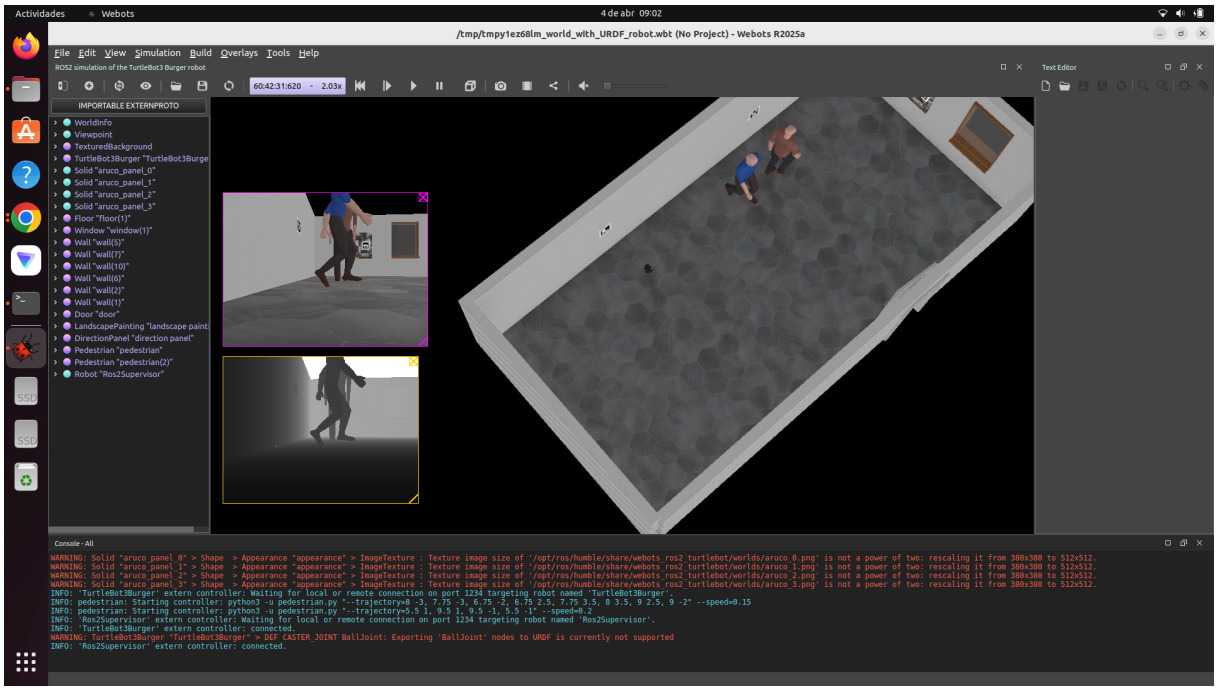

Figure 7: Debug visualisation during tracking. The top-left panel shows the RGB debug image published on `/person_follower/debug_image`: a magenta bounding box surrounds the tracked person, confirming identity lock. The bottom-left panel shows the corresponding `32FC1` depth image from the range finder, where the pedestrian silhouette is clearly visible against the background. The Webots 3-D view (right) confirms the robot is positioned behind and aligned with the target.

argument. These way-points are the exact ground-truth paths and are parsed directly from the world file.

11.2 Reference Frames

The `/odom` topic is expressed in the `odom` frame, whose origin coincides with the robot start pose, whereas the pedestrian way-points are given in the Webots world frame. To overlay both, the odometry samples are transformed into world coordinates with the rigid transformation

$$\begin{pmatrix} x_w \\ y_w \end{pmatrix} = R(\theta_0) \begin{pmatrix} x_o \\ y_o \end{pmatrix} + \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}, \qquad R(\theta_0) = \begin{pmatrix} \cos\theta_0 & -\sin\theta_0 \\ \sin\theta_0 & \cos\theta_0 \end{pmatrix} \tag{22}$$

where $(x_0,\, y_0,\, \theta_0)$ is the robot start pose read from the `.wbt` file. For `turtlebot3_burger_pedestrian_simple.wbt` this is $(x_0, y_0) = (6.75,\, -3.00)$ m and $\theta_0 = \pi/2$ rad. The same transformation is applied to the target topics, since they are computed from the robot odometry and therefore share the `odom` frame.

11.3 Analysis Script

All figures in this section are produced by the script `report/plot_trajectories.py`, which reads a recorded rosbag, parses the world file, applies the transformation above and renders the plots with Matplotlib:

```
# from the report/ directory, with ROS 2 sourced
python3 plot_trajectories.py --bag rosbags/exp_01
```

The procedure to record the rosbag is described in Appendix B.

11.4 Results

A continuous experiment of 10.3 minutes (618 s) was recorded with the robot configured to follow pedestrian 1. During the run the perception pipeline produced 384 valid target detections — an average rate of only 0.62 Hz. The mean interval between detections was 1.6 s and the longest single loss of the target lasted 76 s. This low detection rate is the dominant factor in the results below; its cause is analysed in Section 11.5.

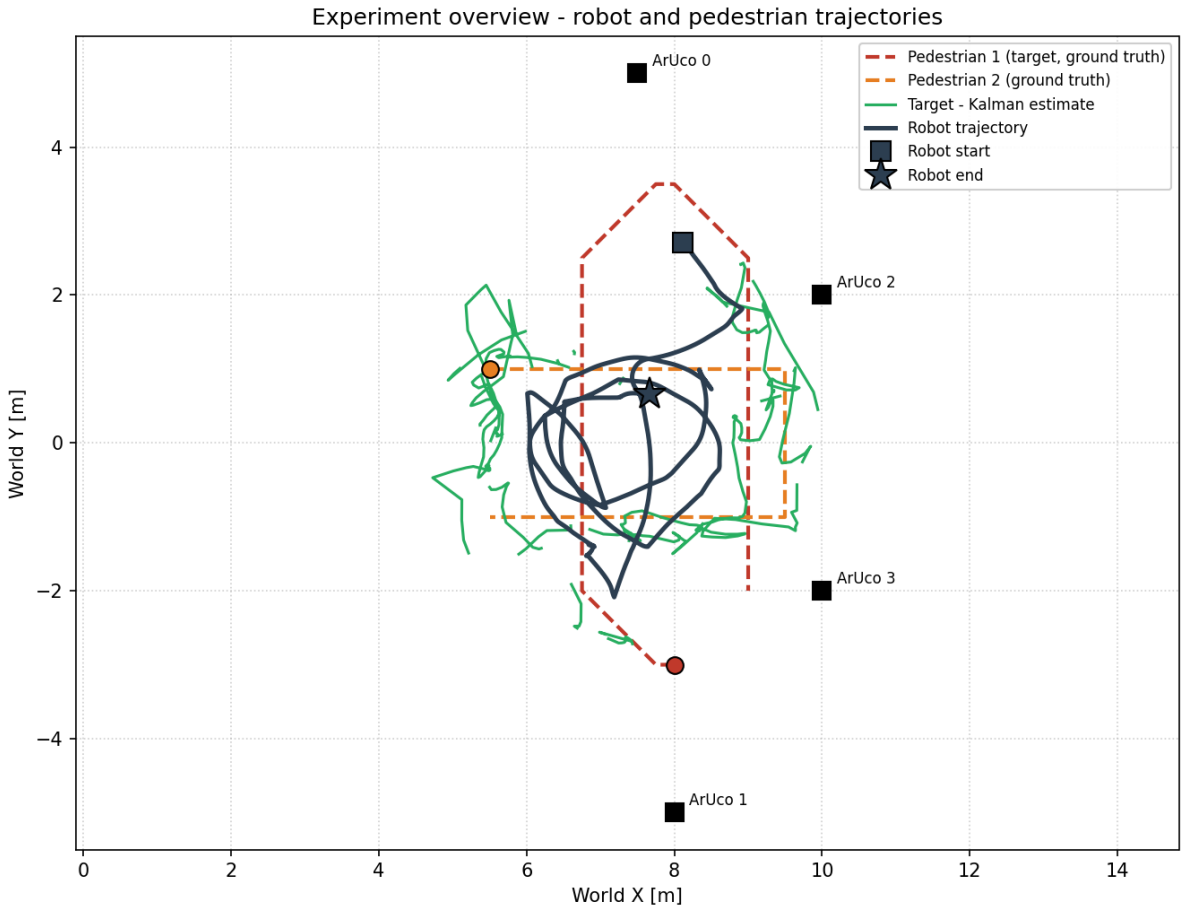

Figure 8: Global overview of the recorded experiment. The robot trajectory is shown in dark blue and the Kalman estimate of the target in green; the green line is broken wherever the target was lost. The ground-truth pedestrian paths are dashed and the four ArUco panel positions are marked for reference.

Figure 8 shows that the robot stayed largely within the central region of the room instead of following pedestrian 1 around its full loop. Because the target was detected only intermittently, the robot spent a large fraction of the run in the search behaviour (Mode 3), turning in place to re-acquire the pedestrian. The Kalman estimate appears as a set of disconnected segments: each gap corresponds to an interval with no detection.

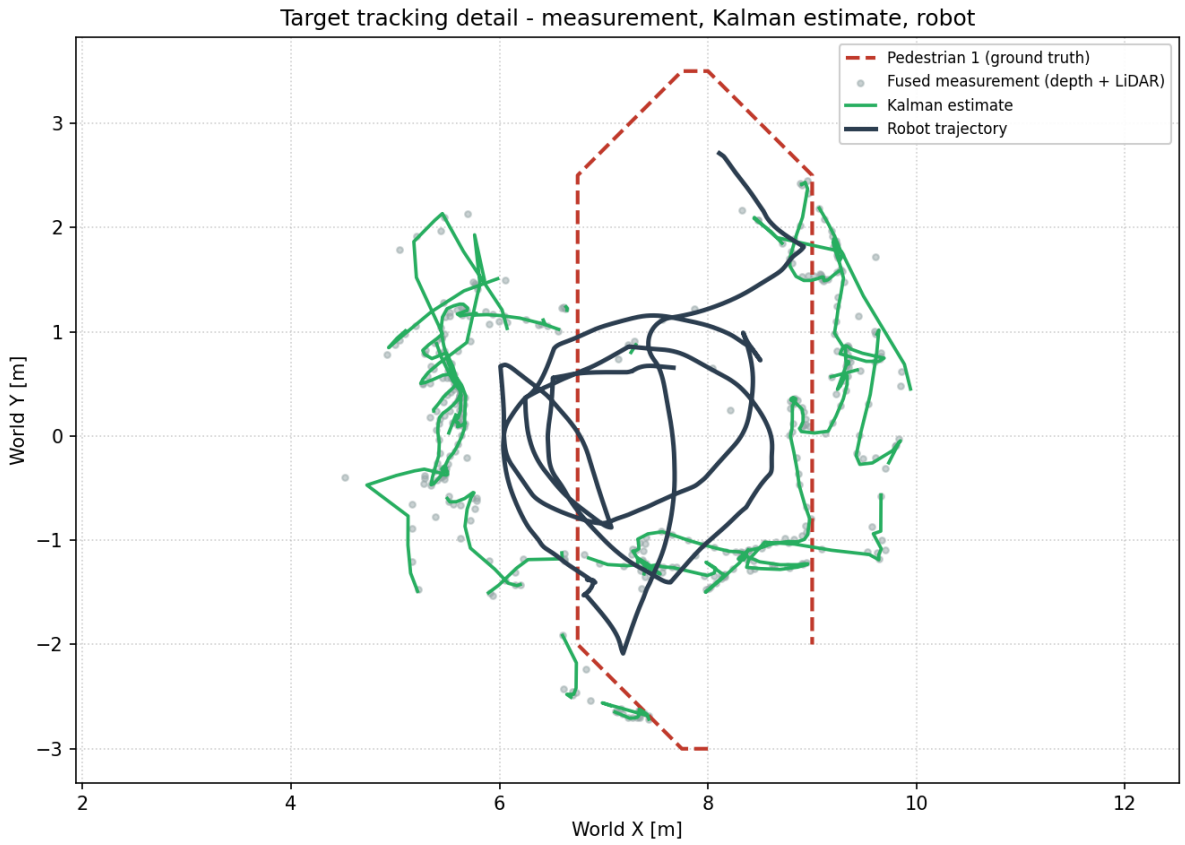

Figure 9: Detail of the target tracking: the fused depth + LiDAR measurements (grey dots), the Kalman estimate (green, broken at detection losses) and the robot trajectory (dark blue), against the ground-truth path of pedestrian 1 (red dashed).

Figure 9 shows that, whenever a detection is available, the fused measurement and the Kalman estimate fall close to the ground-truth pedestrian path: the perception pipeline is accurate. The limitation is not accuracy but *update rate* — the estimate is not refreshed often enough to keep the controller continuously informed.

Figure 10 quantifies the following performance. The robot-to-target distance ranged from 0.56 m to 2.42 m, with a mean of 1.43 m and a median of 1.38 m — well above the desired 0.8 m. Only 27% of the detections fell within the 0.5–1.1 m band around the set-point, although the robot never came dangerously close to the pedestrian (no detection below 0.4 m). The frequent breaks in the line are the detection losses already apparent in the previous figures.

11.5 Discussion and Limitations

The experiment was run on a laptop *without a dedicated GPU*. Running YOLOv8 inference on the CPU is the bottleneck of the whole pipeline: it caps the perception update rate at roughly 0.6 Hz, so the robot is effectively blind for about 1.6 s between detections. At a walking speed of 0.15 m/s the pedestrian moves around 0.25 m in that interval, and after a longer loss it can leave the camera field of view entirely, forcing the robot into the search behaviour. This is the direct cause of the large distance error and the fragmented estimate reported above.

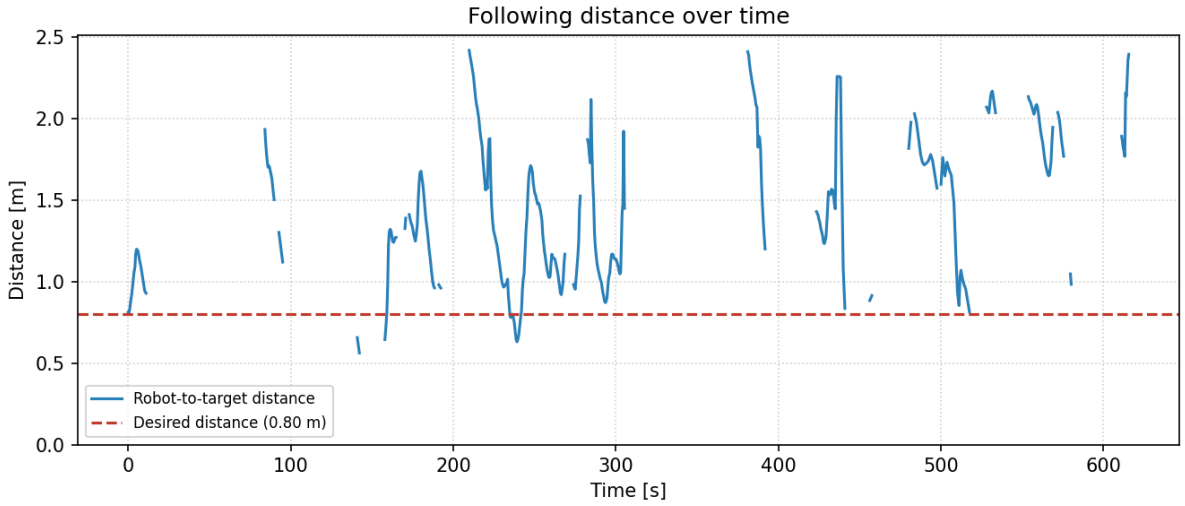

Figure 10: Robot-to-target distance, evaluated at each detection. The line is broken where the target was lost for more than 2 s. The desired following distance of 0.8 m is the red dashed line.

This is a *hardware* limitation rather than a flaw in the design. When a detection is available the fused estimate is accurate (Figure 9), and the sensor-fusion, Kalman-filtering and control logic behave as intended. Continuous following at the designed 0.8 m gap requires a perception rate on the order of 10–30 Hz; on this pipeline that means GPU-accelerated inference. Lighter alternatives — a smaller detector, a lower camera resolution, or running YOLO less often while relying more on the Kalman prediction between frames — would also raise the effective update rate. Re-running the same experiment on a GPU-equipped machine is expected to bring the following distance close to the 0.8 m set-point.

12 Conclusion

The person-follower node integrates five complementary techniques for person tracking in a cluttered indoor simulation:

1. **YOLOv8 + BoTSORT** provide reliable short-term person detection and identity maintenance.
2. **HSV Re-ID** recovers the correct target after occlusions using appearance similarity.
3. **Sensor fusion** (depth + LiDAR) produces a more robust distance estimate than either sensor alone.
4. **Kalman filtering** in world coordinates smooths noisy measurements and enables short-horizon dead-reckoning navigation.
5. **ArUco markers** provide an absolute position reference that can correct odometry drift over long sessions.

The modular callback architecture makes it straightforward to tune individual subsystems independently and to extend the pipeline with additional sensors or more sophisticated controllers.

The trajectory analysis of Section 11 showed, however, that on CPU-only hardware the achievable perception rate — not the control or fusion logic — becomes the factor that limits following performance. GPU-accelerated inference is therefore recommended for real-time operation.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[2] O. Michel, "Webots: Professional Mobile Robot Simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.

[3] R. Amsters and P. Slaets, "Turtlebot 3 as a Robotics Education Platform," in *Robotics in Education (RiE 2019)*, Advances in Intelligent Systems and Computing, vol. 1023. Cham: Springer, 2020, pp. 170–181.

[4] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[7] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *Proc. IEEE Int. Conf. on Image Processing (ICIP)*, 2016, pp. 3464–3468.

[9] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust Associations Multi-Pedestrian Tracking," arXiv:2206.14651, 2022.

[10] D. Comaniciu, V. Ramesh, and P. Meer, "Kernel-Based Object Tracking," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 25, no. 5, pp. 564–577, 2003.

[11] K. O. Arras, O. Martínez Mozos, and W. Burgard, "Using Boosted Features for the Detection of People in 2D Range Data," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2007, pp. 3402–3407.

[12] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

[13] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, "Speeded up detection of squared fiducial markers," *Image and Vision Computing*, vol. 76, pp. 38–47, 2018.

[14] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[15] Y. Bar-Shalom, X.-R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York: John Wiley & Sons, 2001.

A Running the Simulation

This appendix gives the complete procedure to build and run the person-follower in simulation. It assumes Ubuntu 22.04 with ROS 2 Humble [1] and Webots R2025a [2] already installed, with Webots extracted to `$HOME/webots-R2025a`.

A.1 Prerequisites

Install the TurtleBot3 packages for `webots_ros2` and the Python dependencies used by the perception node:

```
sudo apt update
sudo apt install ros-humble-webots-ros2-turtlebot ros-humble-cv-bridge
pip3 install ultralytics opencv-contrib-python numpy
```

A.2 Build the Package

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
git clone https://github.com/mr7g0l/person_follower.git
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
```

A.3 Generate the ArUco Textures

The marker PNG textures are already included in the repository. To regenerate them (for example after changing the marker dictionary), run:

```
cd ~/ros2_ws/src/person_follower
python3 webots/generate_aruco_markers.py
```

A.4 Install the World Files

Copy the world files *and* the ArUco textures into the `webots_ros2_turtlebot` package, so that Webots can locate the texture images referenced by the world:

```
WORLDS=/opt/ros/humble/share/webots_ros2_turtlebot/worlds
sudo cp ~/ros2_ws/src/person_follower/webots/*.wbt       $WORLDS/
sudo cp ~/ros2_ws/src/person_follower/webots/aruco_*.png $WORLDS/
```

A.5 Launch the System

The system runs in three terminals. In every terminal, ROS 2 must be sourced first.

Terminal 1 — perception and control node:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 run person_follower person_follower
```

Terminal 2 — Webots simulator:

```
export WEBOTS_HOME=~/webots-R2025a
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 launch webots_ros2_turtlebot robot_launch.py \
  world:=turtlebot3_burger_pedestrian_simple.wbt
```

Terminal 3 — RViz visualisation (optional):

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
rviz2 -d ~/ros2_ws/src/person_follower/webots/config.rviz
```

Once Webots is running, the robot locks onto the nearest pedestrian and starts following it. The annotated camera stream is published on `/person_follower/debug_image` and can be inspected with `rqt_image_view`.

B Recording and Replaying Experiments

To document an experiment in a reproducible way, the relevant ROS 2 topics are recorded into a rosbag.

B.1 Recording

With the simulation running (Appendix A), start the recording in a new terminal:

```
cd ~/ros2_ws/src/person_follower/webots
./record_experiment.bash exp_01
```

The helper script `record_experiment.bash` stores the bag under `report/rosbags/exp_01_<timestamp>` and records the topics `/odom`, `/cmd_vel`, `/scan`, `/person_follower/target_measurement` and `/person_follower/target_estimate`. Press `Ctrl-C` to stop. The contents of a bag can be inspected with:

```
ros2 bag info report/rosbags/exp_01_<timestamp>
```

B.2 Replaying

A recorded experiment can be played back so that the trajectories, the laser scan and the velocity commands are republished on their original topics:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 bag play report/rosbags/exp_01_<timestamp>
```

While the bag is playing, RViz (Appendix A) can be used to visualise the laser scan and the robot motion.

B.3 Generating the Trajectory Plots

The figures of Section 11 are produced from the recorded bag with:

```
cd ~/ros2_ws/src/person_follower/report
pip3 install matplotlib numpy
python3 plot_trajectories.py --bag rosbags/exp_01_<timestamp>
```

This writes `traj_overview.png`, `traj_following.png` and `traj_distance.png` into the `report/` directory and prints the summary statistics of the run.

B.4 Sharing the Rosbags

Recorded rosbags are too large to be stored in the Git repository (the `report/rosbags/` directory is therefore excluded in `.gitignore`). They are instead uploaded to Google Drive with link sharing enabled, and the link is added here and to the project `README`:

https://drive.google.com/drive/folders/1HjiBTtxrNCbCuHkJ0b46jUU2g8fwfBa-?usp=drive_link

To reproduce the analysis, download the bag from that link and run the commands above on the downloaded directory.